

Leva.Framework.Identity

`Leva.Framework.Identity` is the provider-neutral identity contract layer of ArchFramework. It defines principals, auth sessions, authentication, authorization, audit records, and state-facing principal access used by provider libraries and applications.

Identity exists so applications and provider libraries can share the same identity vocabulary without coupling the framework to a specific login provider, account store, web framework, token format, password system, or external identity service. It contains contracts, small principal values, and framework-owned orchestration services, not concrete account persistence, ASP.NET integration, Google sign-in, JWT handling, or application-specific profile data.

`Leva.Framework.Identity` depends on `Leva.Framework.Core` so identity operations use the same `Result`, `Result<T>`, and `Error` model as the rest of the framework. Core must not depend on Identity, and Engine should not depend on Identity directly; hosts and application access objects should connect identity explicitly at the application boundary.

```
Leva.Framework.Identity
    -> Leva.Framework.Core
```

Project overview

Identity is intentionally provider-neutral. It defines who a principal is, how an auth session is represented, how authentication policies establish an auth session, how authorization policies check requirements, and how security-relevant actions can be audited.

Authentication and authorization are separate concepts. Authentication establishes identity through an `AuthenticationMethod` and `AuthenticationPolicy`. Authorization checks whether an established identity satisfies an `AuthorizationRequirement` through an `AuthorizationPolicy`. `AuthenticationService` and `AuthorizationService` are framework-owned orchestration classes that coordinate policies, auth sessions, results, and audit output.

Auth sessions represent active signed-in principal state. `AuthServiceService` creates, loads, signs out, expires, and deletes auth sessions using `IAuthSessionStore` and `AuthSessionPolicy`. Provider libraries decide where identities and auth sessions are stored. A memory provider can keep them in process, a local provider can use application storage and password hashing, an ASP.NET Core provider can resolve the current auth session from web context, and later Google or JWT providers can plug in without changing this library.

Identity keeps profile data minimal. `Principal` contains principal ID, display name, email, roles, permissions, and claims. Application-specific profile fields such as phone number, address, age, avatar, locale, or preferences should live in application models or provider-specific claims, not in the framework identity model.

Audit records are separate from runtime logs. `AuditEntry` records security-relevant identity actions such as authentication, sign-out, auth session changes, and authorization checks. `IAuditSink` receives those entries without coupling principal audit history to the engine runtime log. Applications can later adapt audit entries to logs, journals, files, databases, or external systems.

Files and classes

Principal model

`PrincipalId` - Identifies one principal independently of provider-specific account IDs. `Principal` - Represents a signed-in principal with display name, optional email, roles, permissions, and claims. `PrincipalClaim` - Represents one typed fact about a principal. `PrincipalRole` - Represents one role assigned to a principal. `PrincipalPermission` - Represents one precise capability assigned to a principal. `IPrincipalStore` - Loads principals from provider-specific stores or account sources.

Auth sessions

`AuthSessionId` - Identifies one active or historical auth session. `AuthSession` - Represents an established auth session with principal, status, creation time, optional expiration time, and optional sign-out time. `AuthSessionStatus` - Describes auth session lifecycle state. `AuthSessionPolicy` - Creates auth session IDs, calculates expiration times, and checks whether auth sessions are expired. `IAuthSessionStore` - Stores and loads auth sessions without exposing provider-specific storage details. `AuthSessionService` - Creates, loads, signs out, expires, and deletes auth sessions. `AuthSessionSource` - Resolves the current auth session from the host or provider-specific execution context.

Authentication

`AuthenticationMethod` - Identifies one provider-defined way to authenticate without predefined framework methods. `AuthenticationRequest` - Represents an authentication attempt with method, name, secret, token, and optional provider properties. `AuthenticationResult` - Represents the result of an authentication attempt, including principal, auth session, success flag, and optional reason. `AuthenticationPolicy` - Authenticates requests for one or more provider-defined methods. `AuthenticationService` - Runs authentication policies, creates auth sessions, signs auth sessions out, and writes audit entries.

Authorization

`AuthorizationRequirement` - Represents one authorization requirement such as a role, permission, claim, or provider-defined rule. `AuthorizationRequest` - Represents an authorization check for a principal, optional auth session, and requirement. `AuthorizationResult` - Represents the result of an authorization check with success flag, requirement, and optional reason. `AuthorizationPolicy` - Evaluates whether an authorization request satisfies one provider-defined policy. `BuiltInAuthorizationPolicy` - Evaluates signed-in, role, permission, and claim requirements from the principal model. `AuthorizationService` - Runs authorization policies and writes audit entries for authorization results.

Audit

`AuditAction` - Describes the security-relevant action recorded by an audit entry. `AuditEntry` - Represents one principal audit record with action, time, optional principal, optional auth session, method, requirement, and reason. `IAuditSink` - Receives principal audit entries without coupling audit history to a concrete log or persistence provider. `MemoryAuditSink` - Thread-safe in-memory audit sink for diagnostics and tests. `NullAuditSink` - Audit sink implementation that intentionally ignores audit entries.

State access and errors

`PrincipalAccess` - State-facing principal capability that resolves the current auth session and performs authorization checks. `PrincipalErrors` - Creates common structured identity errors for missing resources, conflicts, invalid data, unavailable providers, failed operations, unauthorized requests, and forbidden actions.